

Robot Motion Planning

Course Summary

This summary is accompanied by a companion website with interactive visualizations. The textbook *Principles of Robot Motion* (Choset et al.) was used for guidance and figures.

Carlos Sulbarán Fandiño

1 Introduction

Motion planning asks how a robot can move from a start configuration to a goal configuration while avoiding obstacles, and this section lays the conceptual groundwork: define the planning problem, characterize a planner, and use metrics to compare planners.

1.1 The Planning Problem

Piano Mover's Problem — Given a rigid body (e.g. a free-flying polyhedron) and a known set of stationary obstacles, find a collision-free path from q_{start} to q_{goal} .

- **Sofa mover's problem:** piano mover's problem restricted to the plane (body + planar obstacles).
- **Generalized mover's problem:** robot is a set of rigid bodies linked at joints (e.g. a robot arm).

Key idea: instead of tracking every point of the robot, track its **configuration** q — the smallest set of real-valued parameters that fully fixes the position of every point of the robot — and plan in the **configuration space** (C-space) Q (formalized in §2).

Path — Continuous $c : [0, 1] \rightarrow Q$ with $c(0) = q_{start}$, $c(1) = q_{goal}$, $c(s) \in Q_{free} \forall s \in [0, 1]$. Purely geometric/kinematic, no notion of time.

Trajectory — A path parametrized by time, $c : [0, T] \rightarrow Q$, with $c \in C^2$ (twice-differentiable, so velocity/acceleration exist). Possible constraints: smoothness, min. length, min. time. Finding one = **trajectory/motion planning**.

1.2 Characterizing a Planner

Book table 1.1 (pp. 28–30) characterizes the motion planning problem along three independent axes:

Task	Robot	Algorithm
• Navigate	• DoF / C-space	• Optimal?
• Cover	• Kinematic vs. dynamic	• Complexity
• Localize	• Omnidirectional vs. nonholonomic	• Completeness
• Map		• Online/offline
		• Sensor-based?

Task.

- **Navigation:** collision-free motion between two configurations.
- **Coverage:** pass a sensor/tool over every point of a region (demining, mowing, painting).
- **Localization:** use a map + sensor data to find the current configuration.
- **Mapping:** explore/sense an unknown environment.

Robot.

- **DoF / C-space:** the dimension and shape of the robot's configuration space (formalized in §2).
- **Kinematic vs. dynamic:** robot modeled kinematically (velocities as controls) or dynamically (forces as controls).
- **Omnidirectional vs. nonholonomic:** omnidirectional can move instantaneously in any C-space direction; nonholonomic is subject to a velocity constraint (e.g. a car cannot slide sideways).

Algorithm.

- **Optimal?:** whether the planner returns the best solution (e.g. shortest path) or merely a valid one.
- **Complexity:** the algorithm's running time and memory use as a function of problem size.
- **Completeness:**
 - **Complete:** always finds a solution if one exists, else correctly reports failure, in finite time.
 - **Resolution complete:** finds a solution if one exists at a given discretization resolution (e.g. grid-based planners).
 - **Probabilistically complete:** $P(\text{success}) \rightarrow 1$ as $\text{time}/\#\text{samples} \rightarrow \infty$ (basis of PRM, RRT).
- **Online/offline:** **offline** builds the whole plan in advance from a known model, then hands it to an executor; **online** builds the plan incrementally while executing.
- **Sensor-based?:** online and interleaves sensing, computation, action (every Bug algorithm, §3).

Optimality, completeness and computational complexity trade off against each other — demanding one typically costs in the others. Full completeness becomes intractable as DoF grows, motivating the two weaker notions above.

1.3 Performance Metrics

- Path length / execution time / energy consumption
- Planning (pre-processing) time
- Safety — clearance/distance kept from obstacles
- Robustness to disturbances and sensing/actuation noise
- Probability of success (randomized planners)
- Memory requirements and asymptotic complexity in problem size (DoF, #obstacles, ...)

Expensive offline pre-processing (building a roadmap once) pays off when the environment is static and many queries follow — cost amortizes. In high-dimensional or frequently-changing environments this trade flips, motivating sampling-based planners (§8). When a planner cannot compute a full roadmap in advance (high-complexity manipulator), probabilistic/sampling approaches are used instead.

1.4 Contents

The rest of this summary is organized as follows.

- **§2 Configuration Space** introduces degrees of freedom, orientation parametrizations, free space and obstacles, C-space topology, and collision detection.
- **§3 Bug Algorithms** covers sensor-based, map-free local navigation strategies (Bug0, Bug1, Bug2, Tangent Bug).
- **§4 Potential Functions** builds a scalar field from an attractive goal and repulsive obstacles and follows its gradient downhill.
- **§5 Grid-Based Planners** presents the Brushfire distance transform and the Wave-Front planner built on top of it.
- **§6 Roadmaps** covers the visibility graph, GVD, and GVG/HGVG.
- **§7 Cell Decompositions** covers trapezoidal and boustrophedon decomposition, Canny's roadmap algorithm, approximate (quadtree) decomposition, and online/sensor-based coverage.
- **§8 Roadmapping with Random Sampling** covers probabilistically complete planners (PRM, RRT, RRT*, SRT) that scale to high-dimensional C-spaces.
- **§9 Kalman Filter** estimates robot state from noisy sensors: probabilistic estimation, the linear and extended Kalman filter, and Kalman filter-based SLAM.
- **§10 Bayesian Methods** generalizes state estimation to non-Gaussian beliefs via recursive Bayesian filtering, histogram and particle-filter representations, and Bayesian SLAM.

2 Configuration Space

The configuration space (C-space) reframes the planning problem: instead of reasoning about a robot's full physical shape, it treats the robot as a single point moving through the space of all possible configurations.

2.1 Basics

Workspace W : physical ambient space the robot moves through ($\mathbb{R}^2$ or $\mathbb{R}^3$); $W_{free} = W \setminus \bigcup_i WO_i$. For an arm, often taken as the reachable space of the end-effector.

Task space: whatever space the task is most naturally expressed in (e.g. end-effector position).

Configuration q : smallest set of real-valued coordinates fully describing the position of every point of the robot; $q = (\text{position}, \text{orientation}) = (x, y, z, \dots)$.

Configuration space (C-space) Q : space of all configurations the robot can achieve.

Dimension of Q / DoF: minimum # parameters needed to specify q .

These three spaces coincide only for a point robot in the plane; e.g. a 2-link planar arm has a 2D *task* workspace (an annulus) but its C-space is $Q = T^2$ (torus, not $\mathbb{R}^2$), because each joint angle wraps around (S^1 , not $\mathbb{R}$).

Common robot models: single points, single rigid bodies, multiple rigid bodies with joints.

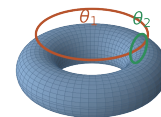


Figure 1: The C-space of a 2R planar arm is a torus $T^2 = S^1 \times S^1$: each joint angle (θ_1, θ_2) independently wraps around a circle, so the two angles together sweep out the surface of a donut, not a flat square (cf. book Fig. 3.10).

Non-holonomic robot: cannot move freely on the C-space manifold in every direction (constrained), e.g. a car.

2.2 Degrees of Freedom

This section covers how to determine a robot's degrees of freedom (DoF) — the dimension of its C-space — using a different counting approach for each kind of mechanism: open chains, single rigid bodies, and closed chains.

Open chains (serial mechanisms). DoF = sum of DoF of each joint: revolute (R) or prismatic (P) joint = 1, spherical (ball-and-socket) joint = 3.

A 2R planar arm has 2 DoF, $Q = S^1 \times S^1 = T^2$.

Single rigid body — points-and-constraints method.

Fix points on the body one at a time, counting remaining freedom after each rigidity (distance) constraint.

- **2D:** point A free in plane (2 DoF); B fixed-distance from $A \Rightarrow$ confined to a circle (1 DoF, angle); any third point already pinned by distances to A, B . Total **3 DoF** (2 translational + 1 rotational).
- **3D:** A free in space (3 DoF); B confined to a sphere about A (2 DoF); C constrained by distances to *both* $A, B \Rightarrow$ confined to a circle about axis AB (1 DoF); every other point now fixed. Total **6 DoF** (3 translational + 3 rotational).

Equivalently from the group structure: the rigid transformations of n -space form the **special Euclidean group** $SE(n)$, which factors as $SE(n) \cong SO(n) \times \mathbb{R}^n$ — a rotation, drawn from the **special orthogonal group** $SO(n)$, composed with a translation in $\mathbb{R}^n$. A rotation $R \in SO(n)$ is an $n \times n$ matrix, so n^2 raw numbers, constrained by $R^T R = I$; since $R^T R$ is symmetric, that matrix equation is really only $n(n+1)/2$ independent scalar constraints (a symmetric $n \times n$ matrix has that many distinct entries), leaving $\dim SO(n) = n^2 - n(n+1)/2 = n(n-1)/2$ free rotational parameters. Adding back the n translational DoF gives the total: a rigid body in n -space has $n(n-1)/2 + n = n(n+1)/2$ DoF overall, of which $n(n-1)/2$ are rotational. Matches: $n = 2 \Rightarrow 3$ DoF, $n = 3 \Rightarrow 6$ DoF.

Grübler's Formula (closed chains / parallel mechanisms) — For k links (incl. 1 fixed ground link, so $k-1$ movable) and n joints:

$$M = N(k-1) - \sum_{i=1}^n (N - f_i) = N(k-n-1) + \sum_{i=1}^n f_i$$

$N = 6$ (spatial/3D) or 3 (planar/2D); $f_i =$ DoF of joint i (1 for R/P, 3 for spherical). Valid only if joint constraints are independent.

Worked example: planar 6-link, 7-revolute-joint mechanism ($N = 3, k = 6, n = 7, f_i = 1 \forall i$):

$$M = 3(6 - 7 - 1) + 7 \cdot 1 = 3(-2) + 7 = 1 \text{ DoF}$$

— despite 6 links and 7 joints looking heavily constrained.

2.3 Parametrizations of Orientation

Representation (#nums)	Advantage	Drawback
Rotation matrix $R \in SO(3)$ (9)	Composes by matrix mult. embeds into $SE(3)$ no singularities	Over-parametrized: 9 numbers, 6 constraints ($R^T R = I$)
Euler angles (α, β, γ) (3)	Minimal: exactly #DoF intuitive (roll/pitch/yaw)	Gimbal lock (singularities) many incompatible conventions hard to concatenate
Unit quaternion $u \in S^3$ (4)	Compact, no singularities, cheap compose/inverse	Over-param. by 1 ($\ u\ = 1$) double-covers $SO(3)$ ($u, -u$ same rotation)

$R \in SO(3)$: $R^T R = I$, $\det(R) = 1$; columns of R are the new coordinate axes expressed in the stationary frame.

Euler (Z-Y-Z or roll-pitch-yaw) construction: $R_{xyz}(\alpha, \beta, \gamma) = R_z(\gamma)R_y(\beta)R_x(\alpha)$ (elementary rotations multiplied, applied right-to-left).

Quaternion (axis n , $\|n\| = 1$, angle θ):

$$u = \left(\cos \frac{\theta}{2}, n_x \sin \frac{\theta}{2}, n_y \sin \frac{\theta}{2}, n_z \sin \frac{\theta}{2} \right)$$

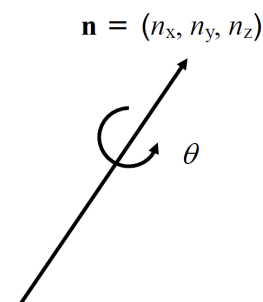


Figure 2: Axis-angle representation: a rotation by angle θ about a unit axis n , the geometric picture behind the quaternion formula above (cf. book Fig. E.2).

Try it live: <https://eater.net/quaternions>

Half-angle $\theta/2$ is what makes quaternion multiplication compose rotations correctly; it is also why a full 360° loop in $SO(3)$ is only a half-loop in S^3 , giving the $u = -u$ double cover.

2.4 Free Space and Obstacles

Obstacles in the workspace induce forbidden regions in C-space; this section defines those regions and describes how to compute them in practice.

$Q = F \cup Q_{obs}$. For workspace obstacle WO_i , the in-

duced **C-space obstacle** is

$$Q_{O_i} = \{q \in Q \mid R(q) \cap WO_i \neq \emptyset\}$$

where $R(q)$ = set of points occupied by the robot at q .

Free space: $F = Q_{free} = Q \setminus \bigcup_i Q_{O_i}$ (open subset of Q).

Obstacle space Q_{obs} : closed subset (contains its boundary).

Semi-free configuration: robot touches an obstacle's boundary but not its interior, i.e. $q \in \text{cl}(F) \setminus F$. A **free** path never touches an obstacle ($c : [0, 1] \rightarrow F$); a **semi-free** path may graze one ($c : [0, 1] \rightarrow \text{cl}(F)$).

Determine free space by discretization (brute force): lay a grid on Q ; for each grid point, place the robot at that exact configuration and run a direct collision test against workspace obstacles, coloring the cell free/occupied. Works for arbitrary robot shapes/DoF, but only *resolution complete* — a path through a gap between grid samples can be missed.

2.5 Topology of Configuration Space

The shape of a C-space, not just its dimension, determines how a planner may reason about it; this section introduces topological vocabulary to describe its shape.

Homeomorphism — Bijection $\varphi : S \rightarrow T$ with φ, φ^{-1} both continuous. S, T then topologically the same (deformable into each other without cutting/gluing).

Diffeomorphism — Smooth bijection φ with smooth φ^{-1} . Strictly stronger (smoothness $\Rightarrow$ continuity, not conversely) $\Rightarrow$ every diffeomorphism is a homeomorphism, not vice versa. Circle $\leftrightarrow$ ellipse: diffeomorphic. Circle $\leftrightarrow$ "racetrack" (two straight sides + semicircular caps): only homeomorphic — curvature is discontinuous at the 4 joints.

Manifold — S is a k -dimensional manifold if *locally* homeomorphic to $\mathbb{R}^k$: every point has a neighborhood topologically like an open patch of $\mathbb{R}^k$ (even if globally curved, e.g. Earth's surface looks locally flat).

Chart: pair (U, φ) , U open in the manifold, φ a diffeomorphism from U onto an open subset of $\mathbb{R}^k$ (a local coordinate system). A single chart usually cannot cover a whole manifold (no single chart covers all of S^1).

Atlas: collection of charts, compatible (" C -related") on overlaps, whose domains together cover Q ; atlas + Q = differentiable manifold.

Submanifold: smooth subset inheriting the ambient space's differentiability, e.g. $S^1 \subset \mathbb{R}^2$, $T^2 \subset \mathbb{R}^3$ (a torus *cannot* be embedded in $\mathbb{R}^2$, hence always drawn as a doughnut in 3D).

2.6 Point Representation

To simplify collision checking, reduce the robot to a **point** and push its physical bulk onto the obstacles — **Minkowski sum**:

For a circular robot of radius r : obstacle boundary offset outward by r , corners rounded into arcs of radius r .

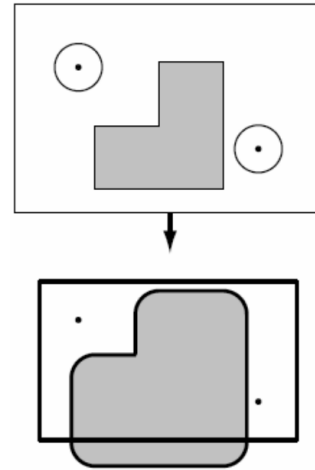


Figure 3: Point representation via Minkowski sum: the robot's bulk is absorbed into an enlarged obstacle, so the robot can now be treated as a single point (cf. book Fig. 3.5).

Exact Minkowski-sum construction stays cheap only for simple shapes at fixed orientation; a rotating robot needs recomputation per orientation, which is why the discretized grid approach (§2.4) is often used instead.

2.7 Collision Detection

Wrap the robot in a cheap **bounding volume**, test that first, fall back to exact geometry only if ambiguous.

- **Circle:** 1 distance calculation (center-to-center vs. sum of radii); loose for elongated/concave (e.g. L-shaped) robots.
- **Axis-aligned rectangle:** up to 4 comparisons; tight only when aligned with robot axes; must be rebuilt on rotation.
- **Hierarchical circle-splitting:** start with one enclosing circle, test. If possible collision, split into smaller child circles covering the robot's parts, recurse (test, split further where ambiguous) down to a minimum radius. — Cheap, refines automatically near actual contact. Best default for irregular shapes.

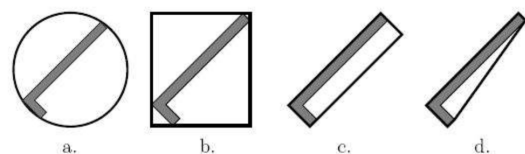


Figure 4: Different hulls require different number of tests (cf. Slides 1.22)

3 Bug Algorithms

The Bug family performs sensor-based, purely local navigation with no map and no pre-processing. Every variant assumes a point robot in the plane with perfect localization (it always knows its own position and the direction and distance to q_{goal}), zero prior knowledge of the obstacles, and either a contact sensor (Bug1/Bug2) or a finite-range radial sensor (Tangent Bug). Each alternates between two behaviors: **motion-to-goal**, a straight line toward q_{goal} , and **boundary-following**, tracing an obstacle's boundary. The **m-line** is the segment from the most recent leave point (or q_{start} , initially) to q_{goal} .

3.1 Bug0

Not in the book.

Strategy: naive.

1. Head towards the goal.
2. When hit point reached, follow the boundary until the direct line to goal is locally unobstructed.
3. Continue from 1.

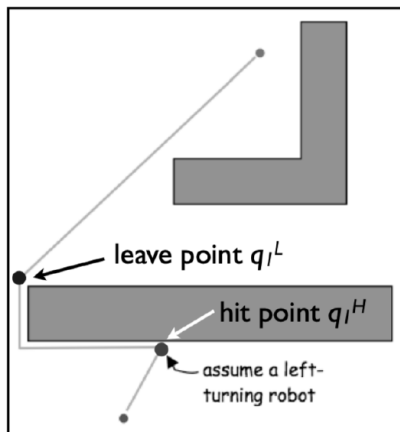


Figure 5: Bug0 example (cf. Slides 2.5)

No termination guarantee: leaves at the *first* opening $\Rightarrow$ can loop forever (spirals, or two obstacles re-triggering each other) (cf. Slides 2.6).

3.2 Bug1

Strategy: exhaustive.

1. Move toward q_{goal} on the m-line.
2. At hit point q_i^H : circumnavigate the *entire* boundary, tracking distance-to-goal throughout.
3. Return to leave point q_i^L = boundary point closest to the goal (shorter retrace direction).
4. Resume motion-to-goal from q_i^L .

5. If the line $q_i^L \rightarrow q_{goal}$ re-intersects the same obstacle: no path exists.

Worst case: $L_{Bug1} \leq d(q_{start}, q_{goal}) + 1.5 \sum_i p_i$, p_i = perimeter of i -th obstacle.

Fails: if q_{start} starts inside a closed obstacle;

Drawback: leave-vector can change the approach direction to the goal (problematic when grasping objects).

3.3 Bug2

Strategy: greedy. m-line is *fixed* for the whole run (not recomputed per obstacle).

1. Move toward q_{goal} on the fixed m-line.
2. At hit point q_i^H : follow the boundary.
3. Leave at a boundary point on the m-line that is closer to the goal than q_i^H . New leave point: q_i^L .
4. Resume motion-to-goal from q_i^L .
5. If the robot re-encounters q_i^H : no path exists.

Worst case: $L_{Bug2} \leq d(q_{start}, q_{goal}) + 0.5 \sum_i n_i p_i$, n_i = # times the m-line crosses obstacle i .

Fails: if q_{start} inside a closed obstacle; also on spiral-shaped obstacles.

Neither dominates: Bug2 wins on simple/convex obstacles (small n_i , no cost for a full lap). Bug1 wins as obstacle complexity (n_i , m-line crossings) grows, since its bound does not depend on n_i at all — it always finds the globally best leave point on the boundary.

Try it live: <https://carloswulba.github.io/Robot-Motion-Planning-Course-Summary/index.html#bug-algorithms>

3.4 Tangent Bug

Tangent Bug extends the Bug idea with a finite-range sensor instead of contact-only sensing. Like Bug1/Bug2 it still alternates between the same two phases, motion-to-goal and boundary-following, but now a look-ahead heuristic, computed purely from whatever the sensor currently sees, decides both phases: while heading for the goal it picks the most promising sensed direction, and while following a boundary it continuously compares what has been seen so far against what is reachable right now to decide the exact instant to switch back to motion-to-goal — rather than committing to a full lap (Bug1) or waiting for a fixed geometric condition on the m-line (Bug2).

Robot has a finite-range (R) radial distance sensor, 360° . Heuristic distance to a sensed point p : $h(p) = d(x, p) + d(p, q_{goal})$. **Motion-to-goal:** move toward the sensed boundary/discontinuity point minimizing $h(p)$, until h stops decreasing (local minimum).

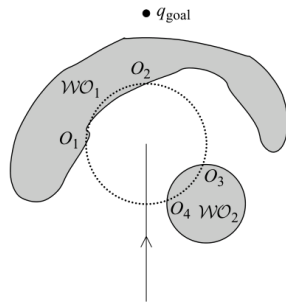


Figure 6: The dotted circle is the robot's sensing range; O_i are points of discontinuity in what the sensor can see. As the robot moves it re-evaluates which direction currently minimizes $h(p)$. (cf. book Fig. 2.7)

Boundary-following: continuously track

- $d_{followed}$: shortest distance-to-goal among boundary points sensed so far on the followed obstacle;
- d_{reach} : best distance-to-goal achievable right now from a point currently within line-of-sight.

Leave condition: as soon as $d_{reach} < d_{followed}$, leave the boundary and resume motion-to-goal.

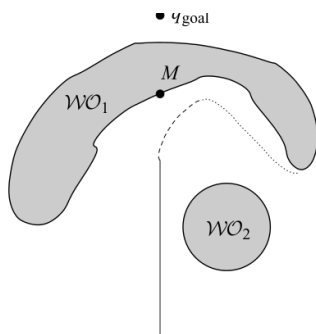


Figure 7: M is where motion-to-goal first gets deflected by the obstacle (a local minimum of the heuristic). The dashed/dotted arc is boundary-following, continuously comparing $d_{followed}$ against d_{reach} (cf. book Fig. 2.10).

$R \rightarrow 0$ (contact-only): the leave condition $d_{reach} < d_{followed}$ collapses to "leave as soon as the current contact point beats every boundary point seen so far" — a purely local record-beating test with no reference to any fixed line, exactly Bug0's rule, not Bug2's (which requires the leave point to lie on the fixed m-line). So Tangent Bug degenerates to **Bug0**, inheriting its lack of a completeness guarantee. $R \rightarrow \infty$: can see past an entire obstacle silhouette to a much better route than boundary-following would ever find.

Try it live: <https://carloshulba.github.io/Robot-Motion-Planning-Course-Summary/index.html#bug-algorithms>

3.5 Comparing the Family

Alg.	Sensor	Search strategy	Path quality
Bug0	contact	naive	no guarantee, may loop
Bug1	contact	exhaustive	predictable, often longer
Bug2	contact	greedy	often shorter, less predictable
TangentBug	finite-range radial	greedy + look-ahead	usually best

Sensor range effect: larger range $\Rightarrow$ can see more of an obstacle's silhouette before committing to boundary-following, giving shorter paths; contact-only sensing forces physical contact before any reaction is possible.

4 Potential Functions

Potential fields treat the goal as an attractive charge and obstacles as repulsive ones, then let the robot follow the resulting field downhill; this section builds that field and shows where the idea breaks.

Idea: scalar $U(q) = U_{att}(q) + U_{rep}(q)$ over F ; robot performs gradient descent, velocity $\propto -\nabla U(q)$. Ideal field: global min. at q_{goal} , no other local minima, $\rightarrow \infty$ near obstacles, smooth.

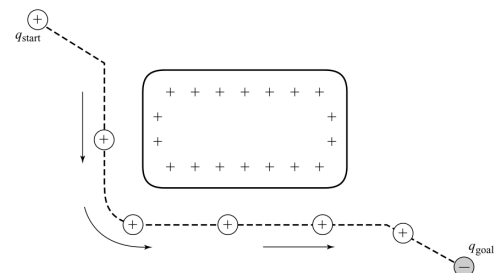


Figure 8: Electrostatic intuition: the goal is a negative charge (attracts), the obstacle boundary a ring of positive charges (repels). The robot is a positive test charge and simply follows the field (cf. book Fig. 4.1).

Attractive.

Conic $U_{att} = \xi d$: bounded gradient, so commanded velocity stays safely capped far from the goal — but the direction is discontinuous at q_{goal} (chatter), and since the gradient magnitude never naturally decreases as the robot nears the goal, the robot doesn't slow down on its own and will overshoot or oscillate around q_{goal} unless something else intervenes.

Quadratic $U_{att} = \frac{1}{2}\xi d^2$: gradient grows linearly $\Rightarrow$ unbounded commanded velocity far away, but decelerates smoothly and stops exactly at the goal.

Standard fix — piecewise conic, matched (C^1) at threshold d_{goal}^* :

$$U_{att}(q) = \begin{cases} \frac{1}{2}\xi d(q, q_{goal})^2, & d \leq d_{goal}^* \\ d_{goal}^*\xi d(q, q_{goal}) - \frac{1}{2}\xi(d_{goal}^*)^2, & d > d_{goal}^* \end{cases}$$

(value *and* gradient agree at $d = d_{goal}^*$: quadratic decelerates smoothly near the goal, linear caps cruising speed far away.)

Repulsive. ρ_0 : influence radius around an obstacle; $D(q)$: distance to nearest obstacle point.

$$U_{rep}(q) = \begin{cases} \frac{1}{2}\eta \left(\frac{1}{D(q)} - \frac{1}{\rho_0} \right)^2, & D(q) \leq \rho_0 \\ 0, & D(q) > \rho_0 \end{cases}$$

$D(q) \rightarrow 0 \Rightarrow U_{rep} \rightarrow \infty$ (blows up at contact); at $D(q) = \rho_0$ value & gradient both $\rightarrow 0$ smoothly (no discontinuity crossing into/out of influence).

- Advantage: cheap gradient evaluations, purely local.
- Disadvantage: **local minima** — opposing obstacles can cancel the attractive pull ($\nabla U = 0$, $q \neq q_{goal}$) $\Rightarrow$ robot stuck; no completeness guarantee; basic form ignores dynamic constraints (required force may exceed robot limits).

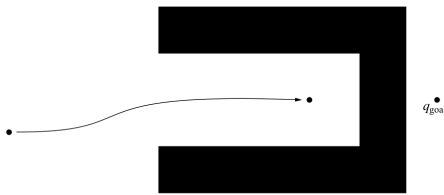


Figure 9: The classic failure case: inside a concave (horseshoe) obstacle, the repulsive gradient from the near wall exactly balances the attractive pull toward q_{goal} well before the robot escapes the concavity (cf. book Fig. 4.10).

Try it live: <https://carlossulba.github.io/Robot-Motion-Planning-Course-Summary/index.html#potential-functions>

5 Grid-Based Planners

Grid-based planners discretize free space into a regular grid and compute a value at every cell. **Assumptions:** the workspace/C-space is known exactly in advance (offline, not sensor-based); it can be discretized into a grid fine enough to matter, with every cell tagged free or occupied; and the robot is modeled as a point (or a fixed footprint) moving between adjacent free cells. **When to prefer them:** over potential fields, when a resolution-complete guarantee matters more than raw speed — a grid-based planner is guaranteed to find a path if one exists at that resolution, whereas a potential field can get permanently stuck in a local minimum;

over the sampling-based planners of §8, when the C-space is low-dimensional enough (typical 2D/3D navigation) that a fully precomputed, exact grid stays cheap rather than needing a probabilistically-complete substitute. This section covers two such planners: the Brushfire distance transform and the Wave-Front planner built on top of it.

5.1 Brushfire Planner

- Discrete grid-based distance transform; generalizes the gradient/potential idea onto cells.
1. **Init:** all obstacle cells *and* outer boundary cells = 1 ("fire" seeded there simultaneously).
 2. **Propagate:** each free cell gets value = $1 + \min(\text{neighbor value})$; repeat (multi-source BFS) until every free cell has a wave value.
- Connectivity choice matters:
 - **4-connectivity:** cheaper; coarser (L_1 /Manhattan-like) distance approximation.
 - **8-connectivity:** costlier; closer to Euclidean distance.
 - Here, two fires from different obstacles meet at an equal value $\Rightarrow$ the point is equidistant from ≥ 2 obstacles $\Rightarrow$ it lies on the Generalized Voronoi Diagram (GVD, §6.2).

Mnemonic, i.e. a memory aid for the relationship above: picture Brushfire as lighting a fire at every obstacle simultaneously and watching it flood outward in expanding rings, one wave per obstacle. The GVD is exactly the set of places where two of those separate waves collide — "*wavefront from obstacles, GVD where two wavefronts meet.*"

Try it live: <https://carlossulba.github.io/Robot-Motion-Planning-Course-Summary/index.html#grid-based-planners>

5.2 Wave-Front Planner

- Wave-Front reuses Brushfire's propagation mechanism but changes where the fire starts. Brushfire ignites every obstacle cell simultaneously; Wave-Front instead seeds a single fire at the *goal* cell (value 2; obstacles = 1) and lets it spread outward alone.
- Propagate wave value $k \rightarrow k+1$ to free neighbors (4- or 8-connectivity) until every reachable free cell has a wave value.
- Every free cell's wave value is now its distance, in moves, to the goal. From *any* start cell, extract a path by walking *downhill* (step to a strictly smaller neighbor wave value) all the way to the goal.
- Seeding at the goal (not the start) is what makes the planner immune to local minima: every free cell already carries a wave value that strictly decreases

toward the goal, so greedy descent from any position is guaranteed to reach it — no smooth gradient needed, just an exhaustive layer-by-layer flood fill from the goal outward.

- Path is shortest only w.r.t. the connectivity metric used: 4-conn. $\rightarrow L_1$ /Manhattan; 8-conn. $\rightarrow$ closer to Euclidean (45°-increment diagonal steps).

Try it live: <https://carlossulba.github.io/Robot-Motion-Planning-Course-Summary/index.html#grid-based-planners>

Curse of dimensionality: every grid-based planner's cost is proportional to its number of cells, which grows like $(\text{resolution})^{\dim Q}$ — exponential in the dimension of Q . A grid fine enough to be useful in 2D or 3D quickly becomes computationally impractical for a 6-DoF arm or higher, which is exactly why the sampling-based planners of §8 exist.

6 Roadmaps

Roadmaps represent the connectivity of free space as a compact network of 1D curves, built once offline and then queried cheaply many times.

Idea: represent connectivity of F as a network of 1D curves.

Query: (1) connect q_{start}, q_{goal} to the network, (2) graph search (Dijkstra/A*).

Pro: built once if obstacle topology is static.

Con: poor in dynamic environments.

6.1 Visibility Graph

- Domain: planar, polygonal obstacles.
- Nodes: q_{start}, q_{goal} , all obstacle vertices.
- Edges: connect two nodes if the straight segment stays outside every obstacle's interior (may run along a boundary).
- Contains the geometric shortest path — but that path grazes obstacle vertices with *zero clearance*.
- Complexity:
 - **Naive:** $O(n^3)$ (all pairs vs. all edges).
 - **Rotational sweep:** $O(n^2 \log n)$.
 - **Optimal algorithm:** $O(n^2)$ time & space (asymptotically tight — graph can have $\Theta(n^2)$ edges).
- **Reduced/simplified visibility graph:** keep only tangent edges between each obstacle pair, drastically fewer edges with shortest-path connectivity preserved:
 - **Supporting lines:** both obstacles on the same

side.

- **Separating lines:** one obstacle per side.

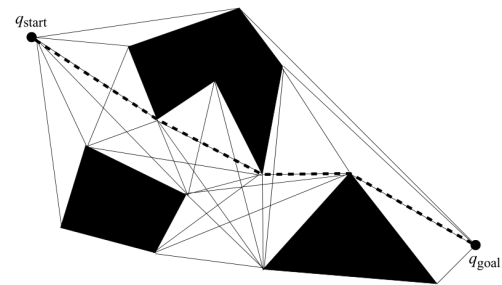


Figure 10: Visibility graph for three polygonal obstacles (thin lines); the dashed line is the shortest path found by graph search between q_{start} and q_{goal} (cf. book Fig. 5.4).

The optimal $O(n^2)$ algorithm, briefly: rotational sweep rebuilds a sorted list of currently-visible edges from scratch at every one of the n vertices, costing $O(n \log n)$ per vertex and $O(n^2 \log n)$ overall. The optimal algorithm instead runs all n sweeps together against one shared structure, updating it incrementally as it goes instead of resorting from zero each time; only $O(n^2)$ individual visibility changes ever happen in total across every sweep, and each is handled in $O(1)$ amortized time once the shared structure is in place, so the extra $\log n$ factor disappears.

Constructing the graph (naive version):

1. Nodes $V \leftarrow \{q_{start}, q_{goal}\} \cup \{\text{obstacle vertices}\}$.
2. For every pair $u, v \in V$: test segment uv against every obstacle edge; if no obstacle edge properly crosses it and no obstacle interior blocks it, add edge (u, v) to the graph.
3. Run Dijkstra/A* on the resulting graph from q_{start} to q_{goal} .

Try it live: <https://carlossulba.github.io/Robot-Motion-Planning-Course-Summary/index.html#roadmaps>

6.2 Voronoi Diagram (GVD)

- Opposite objective to visibility graph: maximize clearance, not minimize length.
- Def: locus of points equidistant to ≥ 2 nearest obstacles.
- Construction via Brushfire: grid init free=0/obstacle=1; propagate distance-to-nearest-obstacle; cells where two distinct waves meet at equal value $\in$ GVD.
- Complexity (continuous construction): $O(n \log n)$ time, $O(n)$ space.

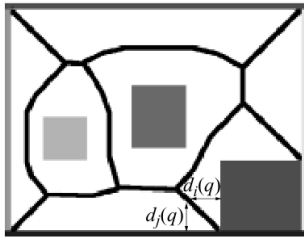


Figure 11: The GVD (thick curves) for a handful of obstacles — every point on a curve is equidistant to (at least) two nearest obstacles (cf. book Fig. 5.9).

Deformation retract — The GVD is a **deformation retract** of Q_{free} , not just an equidistance heuristic: any two connected configurations remain connected via a path lying entirely on the GVD, and every configuration can locally retract onto it (move toward max clearance) without the space's connectivity ever changing. That is why it works as a roadmap.

Try it live: <https://carlossulba.github.io/Robot-Motion-Planning-Course-Summary/index.html#roadmaps>

6.3 GVG / HGVG

The 2D GVD generalizes to higher-dimensional workspaces, but the generalization is more delicate than it first looks, which is why it earns its own name and its own fix.

Generalized Voronoi Graph (GVG): in an m -dimensional workspace, the right analogue of the GVD is the locus of points simultaneously equidistant to m obstacles at once (not just 2). Requiring m -way equidistance is what keeps the result a 1-dimensional *graph* of curves and junctions — the natural shape for a roadmap; requiring only pairwise (2-obstacle) equidistance in $m > 2$ dimensions instead produces an $(m-1)$ -dimensional *surface*, too high-dimensional to serve as a roadmap on its own.

Unlike the 2D GVD (always connected whenever Q_{free} is), the plain GVG in 3D+ is generically *disconnected*: the surfaces of points equidistant to just two obstacles need not intersect at all, so the 1D curves built by demanding m -way equidistance can end up as separate, unconnected pieces even though the underlying free space is one connected region.

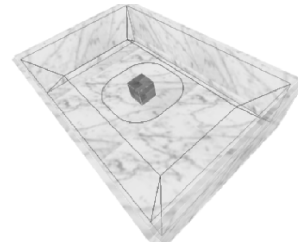


Figure 12: A rectangular room with a box in its interior: the GVG splits into two disconnected pieces — an outer network around the room's walls and an inner "halo" loop around the box — with no edge linking them (cf. book Fig. 5.19).

Hierarchical GVG (HGVG) repairs this by adding a second, coarser layer: on top of the (possibly disconnected) GVG, it adds **second-order edges** — curves equidistant to only 2 obstacles that additionally satisfy one extra distance-matching constraint involving a third obstacle — chosen specifically to bridge the gaps between separate GVG components, stitching everything into a single connected roadmap that the plain GVG cannot guarantee by itself.

Because the HGVG is meant to be traced *online* by a real sensor-equipped robot rather than computed offline from a known map, it is often realized as a control law instead of a precomputed curve.

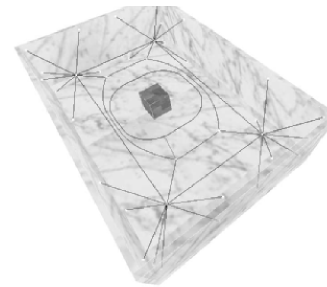


Figure 13: The same room, with the HGVG's second-order edges added — the extra strands radiating out from each junction. These are what bridges the previously separate GVG pieces from the figure above into one connected roadmap (cf. book Fig. 5.20).

7 Cell Decompositions

Cell decomposition instead partitions free space directly into simple, non-overlapping cells connected by an adjacency graph, sidestepping the network-of-curves idea behind roadmaps. **Exact:** cell union = F exactly (no gaps/overlap).

7.1 Trapezoidal Decomposition

- Sweep a vertical line; at every obstacle vertex extend a vertical segment up/down until it hits another edge or the workspace boundary $\Rightarrow$ trapezoids (+ occasional triangles).
- Adjacency graph: 1 node/trapezoid; edge iff cells share a vertical boundary.

- Path: connect q_{start}/q_{goal} to the midpoint of their own cell's vertical extension, search the adjacency graph, chain through extension midpoints via cell interiors.
- Complexity: $O(n \log n)$ time, $O(n)$ space.
- Weakness: a split at *every* vertex over-segments open areas regardless of whether local connectivity actually changes there.

Also called **Vertical Cell Decomposition** in some texts.

Try it live: <https://carlossulba.github.io/Robot-Motion-Planning-Course-Summary/index.html#cell-decompositions>

7.2 Boustrophedon Decomposition

Critical point / Morse decomposition — Boustrophedon is a **Morse decomposition**: insert a boundary only where sweep-line connectivity actually changes (an interval appears/disappears/splits/merges) — a **critical point** — not at every vertex.

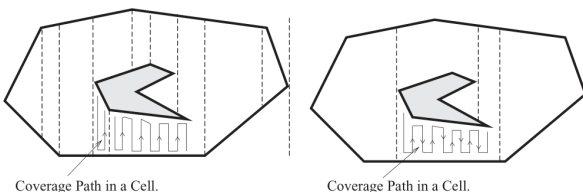


Figure 14: Same free space, two decompositions: trapezoidal (left) slices at every vertex; boustrophedon (right) only at critical points, so the concave obstacle needs fewer cells and a shorter coverage path (cf. book Fig. 6.9).

- Between critical points, a cell is swept back-and-forth $\Rightarrow$ ideal for **coverage** tasks (painting, demining).
- Coarser adjacency graph than trapezoidal — nodes at non-critical vertices are redundant (dropping them loses no reachability, since the merged cell already offers every path they would).

Generalizes to non-polygonal/curved slicing patterns.

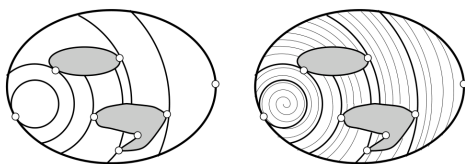


Figure 15: Curved-slice boustrophedon decomposition (cf. book Fig. 6.12).

Try it live: <https://carlossulba.github.io/Robot-Motion-Planning-Course-Summary/index.html#cell-decompositions>

7.3 Cell Decomposition in Higher Dimensions

7.3.1 Canny's Roadmap Algorithm

The great difference between Canny's algorithm and the decompositions above is dimension: trapezoidal and boustrophedon decomposition are inherently 2D geometric sweeps, while Canny's silhouette-sweep construction is dimension-agnostic.

Prefer it when the robot has many degrees of freedom and a guaranteed-complete construction is needed.

The construction itself is not covered here; see book §5.5.1.

7.3.2 The Halting Problem

- **Completeness:** a complete algorithm finds a path if one exists and reports none otherwise.
- Halting problem (proof by contradiction): no algorithm H can decide in general whether an arbitrary program X halts $\Rightarrow$ not all planning algorithms can be complete.
- **Combinatorial** algorithms (cell decomposition, visibility graph, Voronoi diagram) *are* complete — they cover every point of C_{free} exactly $\Rightarrow$ motivates extending them to higher dimensions.

7.3.3 Extension to n Dimensions

Generalizes trapezoidal/boustrophedon:

1. Pick a sweep dimension x_i .
2. Sweep an $(n-1)$ -dim. plane $\perp x_i$; stop & record a new slice at each **critical connectivity event**.
3. Recurse: apply the same sweep-and-slice procedure within each $(n-1)$ -dim. slice.

Result: nested decomposition — n -cells, then $(n-1)$ -cells within those, etc.

7.3.4 k -d Tree

A k -d tree indexes a fixed *point set* (it partitions the points themselves, not F) so that nearest-neighbor and range queries against that set — e.g. finding the milestone closest to a new PRM sample, or the tree node closest to q_{rand} in RRT — can be answered in roughly logarithmic time instead of scanning every point.

- Construction: pick dimension i & a point with coord. x_i ; split points by x_i (greater/less than); recurse on each half, cycling through dimensions with depth.
- Depth $O(\log n)$ if balanced; construction $O(kn \log n)$ ($k = \#$ dimensions).
- Downside: static point set — an environment change may need a substantial rebuild, not a local patch.

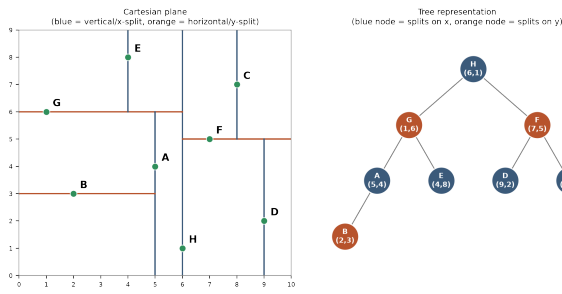


Figure 16: A 2D k-d tree: 8 points split by alternating axis (vertical first, horizontal next, and so on). Left: the resulting partition of the plane. Right: the same splits as a binary tree.

7.4 Approximate Cell Decomposition

While exact decompositions insist on cell union to equal F exactly, approximate cell decompositions trade exactness for a simpler, adaptively-refined grid: it recursively subdivides space only where the map is still ambiguous, leaving large free or occupied regions as single big cells.

- Quadtree (2D) / Octree (3D): recursively split a square/cube cell into 4/8 equal children.
- Stop splitting once a cell is fully free, fully occupied, or a resolution/occupancy threshold is hit (e.g. $\geq 95\%$ occupied).
- Cell union only *under*-approximates F ; mixed boundary cells are left ambiguous — *not* exact.
- Advantage: large open/occupied regions = few big cells (memory savings); a local environment change $\Rightarrow$ only the affected branch is recomputed — cheap, vs. the full rebuild exact decompositions and k-d trees require.

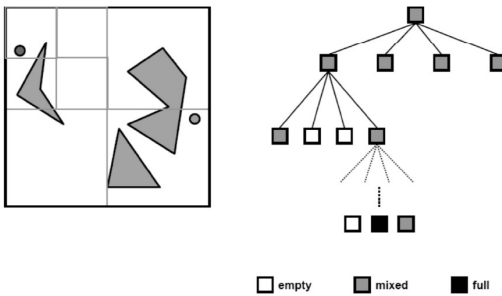


Figure 17: A quadtree recursively splits mixed cells into quadrants until each is fully free/occupied, or a resolution limit is reached (cf. Slides 3.11).

Try it live: <https://carlossulba.github.io/Robot-Motion-Planning-Course-Summary/index.html#cell-decompositions>

8 Roadmapping with Random Sampling

The running time of the classical, exact approaches above grows exponentially with C-space dimension, which is why sampling-based planners instead scale to high-DoF robots. The price is that they give up resolution- or exact-completeness for the weaker **probabilistic completeness**: $P(\text{solution found} \mid \text{exists}) \rightarrow 1$ as $\# \text{samples or time} \rightarrow \infty$.

8.1 Sampling Basics

Correct parametrization per space matters — get it wrong and sample density silently biases the whole downstream planner:

- Unit interval / square / cube: draw $r \in [0, 1]$ i.i.d. per dimension.
- Interval $[a, b]$: $(b - a)r + a$, $r \in [0, 1]$.
- Angle: $\theta \in [0, 2\pi)$ uniform.
- Rotations: representation-dependent. SO(2) = uniform angle directly. SO(3): sample the axis uniformly on the unit sphere, then the angle about it separately (axis-angle).

8.2 Coverage, Connectivity, and Expansiveness

Coverage: milestones distributed so (almost) any free point sees one via a straight line.

Connectivity: every milestone reachable from every other — hardest to guarantee near narrow passages.

$(\epsilon, \alpha, \beta)$ -expansiveness —

- $\text{reach}(x) = \{y \in F \mid \overline{xy} \subset F\}$: the straight-line visibility set of x .
- F is ϵ -good if $\mu(\text{reach}(x)) \geq \epsilon \mu(F)$ for every $x \in F$ — every point sees at least an ϵ -fraction of all of F .
- The β -lookout of a subset $S \subset F$: $\text{lookout}_\beta(S) = \{x \in S \mid \mu(\text{reach}(x) \setminus S) \geq \beta \mu(F \setminus S)\}$ — the points of S that still see a β -fraction of the rest of F .
- F is $(\epsilon, \alpha, \beta)$ -expansive if it is ϵ -good and, for every connected $S \subset F$, $\mu(\text{lookout}_\beta(S)) \geq \alpha \mu(S)$ — an α -fraction of S always has a β -lookout onto the rest of F .

Theorem: in an $(\epsilon, \alpha, \beta)$ -expansive space, sampling

$$n \approx \frac{8}{\epsilon \alpha} \ln \frac{8}{\epsilon \alpha \delta} + \frac{3}{\beta}$$

milestones connects each free-space component with probability $\geq 1 - \delta$. Dominant term scales as $\frac{1}{\epsilon \alpha}$ (log-corrected), plus an additive $\frac{1}{\beta}$ term — depends on *all three* constants, not just one. Smaller ϵ, α, β (more constrained space) $\Rightarrow$ exponentially more milestones needed. A narrow passage $\Leftrightarrow$ small β in that region.

- Limitation: the theorem doesn't say when to *stop* growing the roadmap.
- Improving connectivity: more uniform samples; random-walk resampling; path correction; obstacle-based sampling (OBPRM).

8.3 Multi-Query: PRM

PRM spends time up front building one roadmap that covers the whole free space, then answers many q_{start}/q_{goal} queries against it cheaply — worthwhile whenever the environment is static and will be queried repeatedly.

8.3.1 Basic PRM

1. Sample random configurations; keep collision-free ones as **milestones**.
2. Find neighbors: k -nearest, or all within a fixed radius.
3. Connect candidate pairs with a local planner; add the edge if collision-free.
4. Repeat 1–3 until the roadmap is "dense enough."

Query: connect q_{start}, q_{goal} to the roadmap (same local planner), then graph search.

- Advantages: probabilistically complete; scales to high dimension; fast repeated queries once built.
- Weakness: **narrow passages** — uniform sampling density $\propto$ free-space volume $\Rightarrow$ thin corridors get few/no milestones $\Rightarrow$ disconnected roadmap despite a path existing.

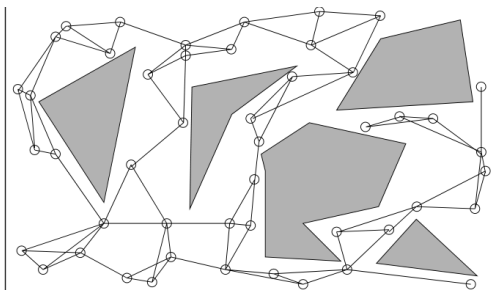


Figure 18: A PRM roadmap for a point robot: circles are collision-free milestones, edges are validated straight-line connections to $k=3$ nearest neighbors (cf. book Fig. 7.3).

Try it live: <https://carlo.sulba.github.io/Robot-Motion-Planning-Course-Summary/index.html#sampling-based>

8.3.2 OBPRM

Obstacle-based sampling — biases milestones toward obstacle surfaces, where narrow passages live:

1. Find a colliding configuration (inside a C-obstacle).
2. Pick a random direction; step along it until reaching a free configuration.
3. Binary-search the segment between the colliding &

free points $\rightarrow$ converge on the boundary; the boundary point becomes the new milestone.

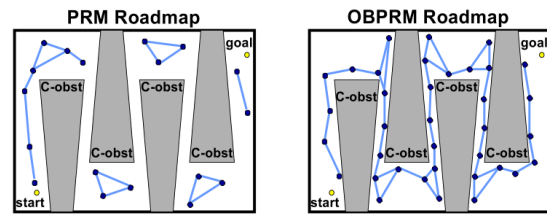


Figure 19: OBPRM pushes samples that start inside an obstacle outward until they land on its boundary, directly seeding milestones where narrow passages live.

8.3.3 Other Sampling Strategies

- **Gaussian sampler:** sample q_1 uniform; q_2 at a step size Gaussian-distributed offset from q_1 ; keep as milestone only if *exactly one* of q_1, q_2 collides.
- **Bridge test:** sample two *colliding* configurations; keep their midpoint as a milestone only if it is collision-free — literally "bridges" a thin gap.
- **GVD-retraction sampling:** retract/push samples onto the GVD — passes through the max-clearance middle of any passage by construction.
- **Random-walk resampling:** on collision, keep wandering in random directions (up to max. length L) instead of discarding the sample outright.
- **Path correction / smoothing:** let a preliminary path cut slightly through obstacles, then push the violating portion back to the nearest free boundary afterward.
- **Visibility-based sampling:** keep a candidate milestone only if it (1) cannot be connected to any existing node or (2) connects ≥ 2 existing components together (cf. book §7.1.3).

Try it live: <https://carlo.sulba.github.io/Robot-Motion-Planning-Course-Summary/index.html#other-sampling-strategies>

8.4 Single-Query Planners

Answer one q_{start}/q_{goal} query fast; no need to cover all of F .

Prefer single-query planners when only one query is needed or when the environment changes often enough that maintaining one big roadmap isn't worth it.

8.4.1 Expansion + Connection (Single-Query PRM)

Grow two trees, rooted at q_{start} and q_{goal} ; alternate **Expansion** and **Connection**:

1. Pick a node x with probability $\propto 1/w(x)$ ($w(x) = \#$ nodes within a fixed radius, incl. x) — favors sparse regions.

2. Randomly sample k points $y_1, \dots, y_k$ around x .
3. Add y_i to the tree with probability related to $1/w(y_i)$ (again favoring sparse regions), provided y_i is collision-free *and* visible from x .
4. If a node in the start tree and a node in the goal tree are close *and* mutually visible: connect them, terminate.

Termination: trees connect, or max. #expansion/connection steps reached.

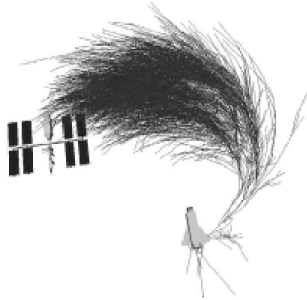


Figure 20: A tree-based planner's search tree for a real spacecraft-docking problem. (Phillips & Kavraki; cf. book Fig. 7.11.)

8.4.2 EST and SBL

EST (Expansive-Spaces Trees): grow tree(s) from q_{start} (optionally a second from q_{goal}).

1. Pick an existing node with probability inversely related to local neighbor density.
2. Sample a candidate configuration near the picked node.
3. Add it *only if* a local planner successfully connects it to the picked node — unlike PRM, which keeps every collision-free sample unconditionally.

Merge two trees by attempting to connect a newly added node in one to its nearest neighbors in the other.

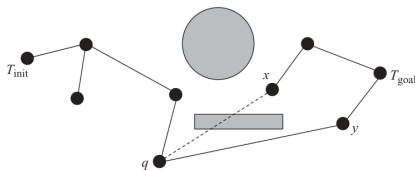


Figure 21: Merging two EST trees: q was just added to T_{init} ; the local planner tries to connect it to its closest configurations x, y in T_{goal} , failing at x but succeeding at y (cf. book Fig. 7.13).

Picking a node can be implemented by overlaying a **coarse grid** on C and picking a sparse, non-empty cell to expand into. This provides a near- $O(1)$ lookup instead of weighing every existing node in each iteration.

SBL (Single-query, Bi-directional, Lazy collision-

checking): two EST-style trees, but *defers all edge collision checks* until an edge becomes part of a candidate start→goal path — saves collision-checking effort on edges that never end up mattering.

Prefer SBL when collision checking is the bottleneck (e.g. high-DoF or complex-geometry robots).

8.5 RRT

The Rapidly-exploring Random Tree is the most common single-query planner in practice, prized for how quickly it explores.

8.5.1 Basic RRT

Grow a tree (uni- or bi-directional, from start/goal) via greedy nearest-extension:

1. Sample random q_{rand} .
2. Find the nearest existing node q_{near} .
3. Extend toward q_{rand} by at most step size Δq .
4. Add the new node if the segment is collision-free; else stop at first collision.

The nearest-neighbor rule makes the tree grow into its largest (least-explored) Voronoi regions $\Rightarrow$ rapid, roughly-uniform exploration as a side effect.

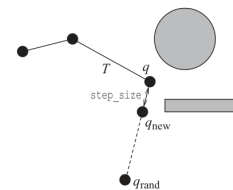


Figure 22: Adding a new configuration to an RRT: q_{rand} is sampled uniformly in Q_{free} ; q is the closest node to it; only q_{new} and (q, q_{new}) are added to the tree (cf. book Fig. 7.14).

Bias toward q_{goal} (e.g. 5% of the time) can help find a path faster.

Try it live: <https://carlosuba.github.io/Robot-Motion-Planning-Course-Summary/index.html#sampling-based>

8.5.2 Kinodynamic RRT

For systems with differential / nonholonomic constraints (e.g. a car): extend by **forward-simulating** a short trajectory of the robot's actual equations of motion (control inputs applied over a small Δt) from q_{near} , instead of a straight line.

The tree then contains dynamically-feasible motions. Standard PRM/RRT (straight-line local planner) cannot be used directly here.

8.5.3 RRT*

Plain RRT is *not* asymptotically optimal — the first path found is kept as-is, never improved. RRT* adds a

rewiring step on every new sample q_{new} :

1. Examine existing nodes within a shrinking radius around q_{new} — not just the single nearest node.
2. **Choose best parent:** connect q_{new} to whichever nearby node gives the lowest cost-from-root (not necessarily the geometrically nearest one).
3. **Rewire:** for each of those nearby nodes, re-parent it through q_{new} if that would lower its own cost-from-root.

As $n \rightarrow \infty$, cost from root to any node converges to the optimal cost $\Rightarrow$ **asymptotically optimal**.

Prefer plain RRT when any valid path is fast enough;
Prefer RRT* when path quality matters and there is time/samples to spare.

Try it live: <https://carlossulba.github.io/Robot-Motion-Planning-Course-Summary/index.html#sampling-based>

8.6 Sampling-Based Roadmap of Trees

SRT combines PRM with tree planners: it builds a PRM-style roadmap whose nodes are not single configurations but whole *trees*, each grown by a single-query planner like EST or RRT.

1. **Add trees:** sample a root uniformly in F , then grow a tree from it with a sampling-based tree planner (EST/RRT).
2. **Add edges:** for each tree T_i , find its closest and a few random neighboring trees; attempt to connect T_i to each via the same tree planner, growing both trees further if needed. Two trees merge into one roadmap component as soon as any configuration in one connects to any configuration in the other.

Prefer SRT on problems so difficult that a single-query planner would need one enormous tree to connect q_{start} to q_{goal} directly.

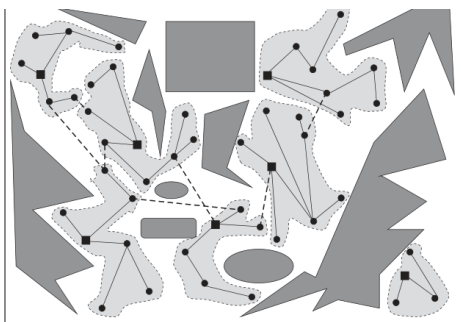


Figure 23: An SRT roadmap: each light-gray blob is one tree (black square = root); dashed lines are edges that merged two trees together (cf. book Fig. 7.16).

8.7 Path Post-Processing

Homotopic paths: two paths with the same endpoints are homotopic if one can be continuously deformed into the other without leaving F . Matters because it fixes which "side" of obstacles a smoothed path passes on relative to the original.

- **Smoothing / shortcutting:** remove zig-zag from a sampling-based path by directly connecting mutually-visible, non-consecutive nodes, or via energy-based nonlinear optimization.
- **PRM path repair:** if an extracted path (or edge) turns out infeasible, locally resample / binary-search push it back into F near the failure point, rather than rebuilding the whole roadmap.

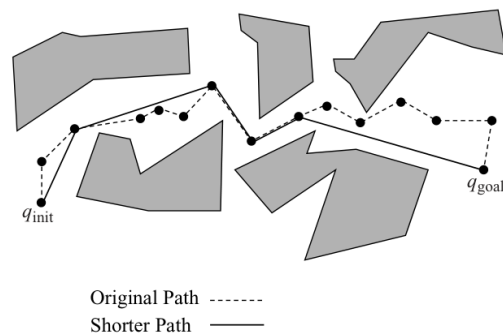


Figure 24: Greedy shortcutting of a PRM path: starting from q_{init} , repeatedly try to connect straight to the current target, falling back one node at a time until a connection succeeds (cf. book Fig. 7.7).

9 Kalman Filter

Every planner so far has assumed either an exact map or perfect sensing. This chapter drops that assumption: the robot's knowledge of its own state now comes from imperfect, noisy measurements, and the job is to estimate that state as well as possible from them.

9.1 Probabilistic Estimation

Instead of maintaining a single hypothesis of where the robot is, **probabilistic estimation** maintains an entire probability distribution over possible states, updated in a two-step cycle:

- **Prediction:** propagate the current belief forward through the motion model. Since the model is imperfect, this step always *loses* information — the distribution spreads out.
- **Update:** fuse the predicted belief with a new sensor reading. This step *gains* information — the distribution sharpens.

Why not just integrate commands (**dead reckoning**)? Because errors accumulate unboundedly and the estimate eventually becomes meaningless.

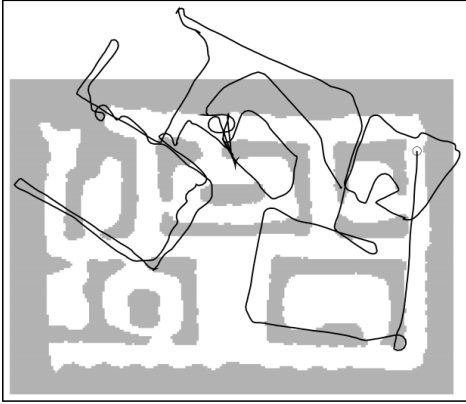


Figure 25: Real odometry trace of a mobile robot (thin curve) against the room it actually drove around (gray floor plan): pure dead reckoning drifts far enough that the trace no longer resembles the room at all — exactly the unbounded error growth the notebook above warns about (cf. book Fig. 8.1).

The Kalman filter is the special case of this idea where the belief is always a **Gaussian** (mean $\hat{x}$, covariance P) — more general, non-Gaussian versions are the subject of §10.

9.2 Linear Kalman Filter

Assume a linear discrete-time system with zero-mean white Gaussian noise:

$$x(k+1) = F(k)x(k) + G(k)u(k) + v(k)$$

$$y(k) = H(k)x(k) + w(k)$$

- x : state;
- u : control input (e.g. commanded velocity);
- y : sensor output;
- v : process noise;
- w : measurement noise.
- $F(k)$: dynamics of the system.
- $G(k)$: how the control input affects the state.
- $H(k)$: how the state is mapped to the sensor.

Where the gain comes from: fusing two noisy estimates z_1 (variance R_1) and z_2 (variance R_2) of the same quantity optimally gives $\hat{r} = z_1 + K(z_2 - z_1)$ with $K = R_1/(R_1+R_2)$ — an inverse-variance-weighted average, trusting whichever estimate is less uncertain. The Kalman gain R below is exactly this idea generalized to vectors: S plays the role of total uncertainty (R_1+R_2), and PH^T

projects it into the state's own units.

Kalman Filter — Prediction:

$$\hat{x}(k+1|k) = F\hat{x}(k|k) + Gu(k)$$

$$P(k+1|k) = FP(k|k)F^T + V$$

Update:

$$\nu = y(k+1) - H\hat{x}(k+1|k)$$

$$S = HP(k+1|k)H^T + W$$

$$R = P(k+1|k)H^T S^{-1}$$

$$\hat{x}(k+1|k+1) = \hat{x}(k+1|k) + R\nu$$

$$P(k+1|k+1) = P(k+1|k) - RHP(k+1|k)$$

- $\hat{x}(k|k)$, $P(k|k)$: **posterior** — state estimate and its error covariance at step k , i.e. *after* incorporating $y(k)$.
- $\hat{x}(k+1|k)$, $P(k+1|k)$: **prior** — the same, *predicted* one step ahead before seeing $y(k+1)$.
- ν : **innovation** — how far the actual measurement $y(k+1)$ landed from what was predicted, $H\hat{x}(k+1|k)$.
- S : **innovation covariance** — how much spread to expect in ν : the predicted state's own uncertainty plus the sensor's noise W .
- R : **Kalman gain** — how much of ν to actually apply to correct the estimate, per state dimension. A "large" R means the sensor reading is more reliable than the prediction; a "small" R means the opposite.

Worked example: robot moving on a line.

State $x = [p, v]^T$ (position, velocity), $\Delta t = 1$, no control input, a sensor that reads position only:

$$F = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad H = [1 \quad 0]$$

Start with $\hat{x}(0|0) = [0, 1]^T$, $P(0|0) = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.1 \end{bmatrix}$, process

noise $V = \begin{bmatrix} 0.01 & 0 \\ 0 & 0.04 \end{bmatrix}$, measurement noise $W = 0.25$, and observe $y(1) = 1.3$.

Prediction:

$$\hat{x}(1|0) = F\hat{x}(0|0) = [1, 1]^T$$

$$P(1|0) = FP(0|0)F^T + V = \begin{bmatrix} 0.21 & 0.1 \\ 0.1 & 0.14 \end{bmatrix}$$

Update:

$$\nu = y(1) - H\hat{x}(1|0) = 1.3 - 1 = 0.3$$

$$S = HP(1|0)H^T + W = 0.21 + 0.25 = 0.46$$

$$R = P(1|0)H^T S^{-1} = [0.457, 0.217]^T$$

$$\hat{x}(1|1) = \hat{x}(1|0) + R\nu = [1.137, 1.065]^T$$

$$P(1|1) = P(1|0) - RHP(1|0) = \begin{bmatrix} 0.114 & 0.054 \\ 0.054 & 0.118 \end{bmatrix}$$

Position moved from 1 (pure prediction) most of the way to the measurement 1.3, since R 's first entry (0.457) is fairly large; velocity also nudged up ($1 \rightarrow 1.065$) purely through the p - v correlation carried in P . And $P(1|1)$ is tighter than $P(1|0)$ in every entry — "update sharpens the belief".

Observability: the pair (F, H) must be observable for $\hat{x}$ to actually converge to the true state — i.e. enough of the state must eventually affect some output.

Prefer the linear KF when the motion and sensor models really are linear (or close enough): it is cheap, closed-form, and exactly optimal.

9.3 Extended Kalman Filter

Most real robot models are nonlinear:

$$x(k+1) = f(x(k), u(k), k) + v(k)$$

$$y(k) = h(x(k), k) + w(k)$$

The **Extended Kalman Filter (EKF)** linearizes about the current estimate at every step, then reuses the same prediction/update equations with the Jacobians

$$F(k) = \left. \frac{\partial f}{\partial x} \right|_{\hat{x}(k|k)}$$

$$H(k+1) = \left. \frac{\partial h}{\partial x} \right|_{\hat{x}(k+1|k)}$$

the rest of the formulas in §9.2 are unchanged.

The key point: $F(k)$ and $H(k+1)$ are no longer fixed matrices — they are recomputed *at every step*.

Because f and h are only *approximated* by their Jacobians, propagating a Gaussian through the true nonlinear function no longer yields an exact Gaussian — FPF^T only matches the first-order (linear) term of the true transformation, discarding higher-order curvature. The EKF is therefore a consistent but *sub-optimal* approximation, unlike the linear KF which is exactly optimal.

Error-state (indirect) EKF: an alternative formulation that propagates the small *error* $\tilde{x} = x - \hat{x}$ between true and estimated state rather than the absolute state itself. For orientation components this lets small-angle approximations ($\cos \tilde{\theta} \approx 1$, $\sin \tilde{\theta} \approx \tilde{\theta}$) keep the Jacobians simple. Mathematically equivalent to the direct form above, but often the more natural choice when the state includes an angle.

Worked example: range-bearing localization.

State $x = [x_r, y_r, \theta_r]^T$; known landmarks at (x_{ℓ_j}, y_{ℓ_j}) ; each measurement gives range and bearing to one landmark,

$$h_j(x) = \begin{bmatrix} \sqrt{(x_r - x_{\ell_j})^2 + (y_r - y_{\ell_j})^2} \\ \text{atan2}(y_r - y_{\ell_j}, x_r - x_{\ell_j}) - \theta_r \end{bmatrix}$$

Differentiating h_j w.r.t. (x_r, y_r, θ_r) , at range r to the landmark:

$$H_j = \begin{bmatrix} -\frac{x_{\ell_j} - x_r}{r} & -\frac{y_{\ell_j} - y_r}{r} & 0 \\ \frac{y_{\ell_j} - y_r}{r^2} & -\frac{x_{\ell_j} - x_r}{r^2} & -1 \end{bmatrix}$$

Landmark at $(3, 4)$, current estimate $\hat{x} = (0, 0, 0)$ — a 3-4-5 triangle, so $r = 5$ — with $P = \text{diag}(0.05, 0.05, 0.02)$ and sensor noise $W = \text{diag}(0.04, 0.01)$. Predicted measurement:

$$h(\hat{x}) = [5, \text{atan2}(4, 3)]^T = [5, 0.927]^T$$

Plugging $r = 5$, $(x_{\ell} - x_r, y_{\ell} - y_r) = (3, 4)$ into H_j above:

$$H = \begin{bmatrix} -0.6 & -0.8 & 0 \\ 0.16 & -0.12 & -1 \end{bmatrix}$$

A real reading $y = [5.2, 0.90]^T$ gives innovation $\nu = y - h(\hat{x}) = [0.2, -0.027]^T$. Running H, ν through exactly the same S, R formulas as §9.2 (with W in place of the linear KF's scalar noise) yields

$$\hat{x}_{new} = [-0.074, -0.084, 0.017]^T$$

— a small correction in *every* state dimension from a single sighting of *one* landmark, because H couples range/bearing error back into x_r, y_r, θ_r all at once.

Data association: if it isn't known *which* landmark a measurement belongs to, compute the innovation ν_{ij} against every candidate landmark j , weight it by its own uncertainty via the Mahalanobis statistic $\chi_{ij}^2 = \nu_{ij} S_{ij}^{-1} \nu_{ij}^T$, and assign the measurement to whichever landmark minimizes χ_{ij}^2 . Range-only localization is the same construction with the bearing row simply dropped from h and H .

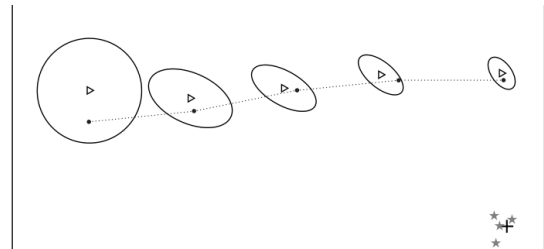


Figure 26: EKF localization along a real path (dotted): each ellipse is the estimate's uncertainty at a landmark sighting, shrinking as more measurements are fused in (cf. book Fig. 8.8).

Prefer the EKF over the linear KF whenever the motion or sensor model is nonlinear — true for almost every real mobile robot. For strongly nonlinear or multimodal problems, prefer a particle filter (§10.2) instead.

9.4 Kalman Filter for SLAM

The key idea: augment the state vector to hold *both* the robot pose *and* every landmark position, then estimate the whole thing with one filter. Landmarks are static, so their block of F is simply the identity.

- **Simple SLAM:** linear case — the robot measures its relative displacement to each (uniquely identified) landmark directly, so $y_i(k) = [x_{\ell_i} - x_r, y_{\ell_i} - y_r]^T(k) + w_i(k)$ is already linear in the augmented state; plug into the linear KF equations directly.
- **Range and bearing SLAM:** the more realistic case — same augmented-state idea, but h is nonlinear.

The joint covariance is block-structured, robot block first:

$$P = \begin{bmatrix} P_{RR} & P_{RL} \\ P_{RL}^T & P_{LL} \end{bmatrix}$$

P_{RR} : robot pose covariance. P_{LL} : landmark-position covariance (one block per landmark, growing with map size). P_{RL} : robot-landmark cross-covariance — as landmarks are observed repeatedly, this off-diagonal block fills in, correlating the robot's pose estimate with every landmark estimate. This is exactly the mechanism that lets a later sighting of an old landmark retroactively sharpen the robot's own pose estimate (loop closure) — but it is also why full KF-SLAM's covariance update scales quadratically in the number of landmarks, unlike plain localization against known landmarks.

Prefer KF/EKF-SLAM when the landmark count is modest (so the $O(n_\ell^2)$ covariance growth stays manageable) and the belief stays close to unimodal; for large-scale or highly ambiguous mapping problems, prefer the particle-filter/optimization-based methods of §10.3.

9.5 Unscented Kalman Filter

The EKF's Jacobian linearization is only *first-order* accurate, and breaks down under strong nonlinearity or when f, h are hard (or impossible) to differentiate. The **Unscented Kalman Filter (UKF)** sidesteps linearization entirely: instead of approximating the *function*, it approximates the *distribution* with a small set of deterministic sample points, then pushes every point through the true, unmodified f or h .

Unscented Transform (UT): for an n -dimensional Gaussian $(\hat{x}, P)$, pick $2n+1$ **sigma points** that exactly reproduce its mean and covariance, propagate each one through the true nonlinear function, then reconstruct a Gaussian from the transformed points' weighted mean/covariance. This matches the true mean and covariance to second order in a Taylor expansion — one order better than the EKF's first-order Jacobian, at a cost of $2n+1$ function evaluations instead of one Jacobian.

Unscented Transform — Sigma points ($\lambda = \alpha^2(n+\kappa) - n$):

$$\chi_0 = \hat{x}, \quad \chi_i = \hat{x} \pm \left(\sqrt{(n+\lambda)P} \right)_i$$

Weights:

$$W_0^{(m)} = \frac{\lambda}{n+\lambda}, \quad W_0^{(c)} = \frac{\lambda}{n+\lambda} + (1-\alpha^2+\beta)$$

$$W_i^{(m)} = W_i^{(c)} = \frac{1}{2(n+\lambda)}, \quad i = 1, \dots, 2n$$

- χ_i : the $2n+1$ sigma points; $\pm$ ranges over each of the n columns of the matrix square root, giving n points on each side of $\hat{x}$.
- α : spread of the sigma points around $\hat{x}$ (small, e.g. 10^{-3}). κ : secondary scaling, usually 0. β : encodes prior knowledge of the distribution's shape ($\beta=2$ is optimal for a Gaussian).
- $W^{(m)}$: weights for recombining the mean; $W^{(c)}$: weights for recombining the covariance (differ only at $i=0$).

UKF algorithm: Prediction — generate sigma points from $(\hat{x}(k|k), P(k|k))$, push each through f , recombine with the weights above to get $(\hat{x}(k+1|k), P(k+1|k))$. **Update** — generate new sigma points from the predicted $(\hat{x}(k+1|k), P(k+1|k))$, push each through h , recombine to get the predicted measurement and its covariance S , plus the state-measurement cross-covariance; then apply the same gain-and-correct step as the linear KF ($R = P_{xy}S^{-1}$), but with these sigma-point-derived quantities in place of HPH^T and PH^T .

Prefer the UKF over the EKF when f or h is strongly nonlinear, discontinuous, or inconvenient to differentiate (no Jacobians needed at all); it is more accurate and often not much more expensive, though still cheaper than a particle filter (§10.2), which needs far more than $2n+1$ samples for comparable accuracy.

10 Bayesian Methods

The Kalman filter is really just one special case of a more general recipe — recursive Bayesian filtering — that works for *any* probability distribution, not only Gaussians. This chapter develops the general framework and two alternative ways to represent a belief that isn't Gaussian.

10.1 Recursive Bayesian Filtering

Let $\text{bel}(x(k)) = P(x(k) \mid u(0:k-1), y(1:k))$ denote the **belief**: the posterior probability of every state $x(k)$, given every movement and measurement seen so far. $\bar{\text{bel}}(x(k))$ is the **predicted belief** — the same idea, but only propagated through the motion model, *before* folding in $y(k)$ — the Bayesian analogue of the Kalman filter's prior $\hat{x}(k+1|k)$ from §9.2.

Two assumptions make this *recursion* possible at all — i.e. let $\text{bel}(x(k))$ be computed from only $\text{bel}(x(k-1))$ instead of the entire history: **Markov assumption** — $x(k)$ depends only on $x(k-1)$ and $u(k-1)$, not on any earlier state; **measurement in-**

dependence — $y(k)$ depends only on the current $x(k)$, not on past states or measurements.

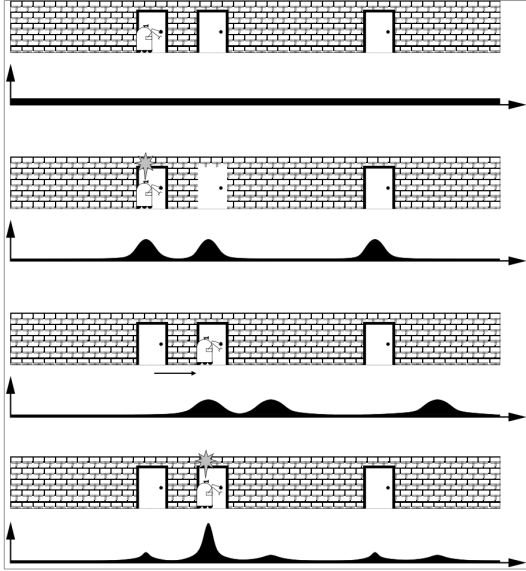


Figure 27: Global localization in a hallway with 3 doors: the belief starts uniform; sensing a door splits it into 3 equally-likely peaks; moving right predicts-forward and flattens each peak slightly; sensing a second door reinforces only the peak consistent with *both* sightings, leaving the robot confident of its true location — prediction spreads the belief out, update sharpens it (cf. book Fig. 9.1).

Bayes Filter — Prediction:

$$\begin{aligned}\bar{\text{bel}}(x(k)) &= \sum_{x(k-1)} P(x(k) \mid u(k-1), x(k-1)) \\ &\quad \times \text{bel}(x(k-1))\end{aligned}$$

Update:

$$\text{bel}(x(k)) = \eta P(y(k) \mid x(k)) \bar{\text{bel}}(x(k))$$

- $P(x(k) \mid u(k-1), x(k-1))$: **motion model** — probability of landing in state $x(k)$, given the previous state and the command that was issued.
- $P(y(k) \mid x(k))$: **sensor model** — how likely the actual reading $y(k)$ is, *if* the robot were really at $x(k)$.
- η : normalizer so $\text{bel}(x(k))$ sums to 1; concretely $\eta = 1 / \sum_{x(k)} P(y(k) \mid x(k)) \bar{\text{bel}}(x(k))$.

Computing $P(y(k) \mid x(k))$ in practice: simulate the reading the robot would *expect* if it were re-

ally at candidate state $x(k)$, using the known map (e.g. ray-cast against walls for a range sensor), then score how well the actual reading matches that expectation under an assumed noise model (Gaussian around the expectation, or an empirical histogram). This is what turns the abstract "sensor model" above into a concrete number.

Extracting a point estimate: once $\text{bel}(x(k))$ is computed, two standard choices reduce it to a single answer: **MAP** (maximum a posteriori) takes $\arg \max_x \text{bel}(x)$; **MMSE** takes the mean $\mathbb{E}[x]$, minimizing expected squared error.

Worked example: a robot in a circular 3-cell hallway; cell 2 has a door.

Motion model: a "move right" command succeeds with probability 0.8, the robot stays put with probability 0.2.

Sensor model: $P(\text{door} \mid x=2) = 0.8$, $P(\text{door} \mid x \neq 2) = 0.2$.

Start from $\text{bel}(x(k-1)) = [0.5, 0.3, 0.2]$ and issue "move right."

Prediction:

$$\bar{\text{bel}}(1) = 0.2(0.5) + 0.8(0.2) = 0.26$$

$$\bar{\text{bel}}(2) = 0.8(0.5) + 0.2(0.3) = 0.46$$

$$\bar{\text{bel}}(3) = 0.8(0.3) + 0.2(0.2) = 0.28$$

Update (sensor reads "door"): first weight each predicted cell by how likely that reading is *if the robot were there*, $P(\text{door} \mid x=i) \bar{\text{bel}}(i)$:

$$0.2(0.26) = 0.052$$

$$0.8(0.46) = 0.368$$

$$0.2(0.28) = 0.056$$

then normalize so the three weighted values sum to 1:

$$\eta^{-1} = 0.052 + 0.368 + 0.056 = 0.476$$

$$\eta = 1/0.476 = 2.101$$

$$\text{bel}(x(k)) = \eta \cdot [0.052, 0.368, 0.056]$$

$$= [0.109, 0.773, 0.118]$$

Prediction alone already favored cell 2 slightly, since moving right from a prior centered on cell 1 lands there most often; sensing a door then collapses most of the remaining mass onto it.

10.2 Representing the Belief

Repr.	Scales w/ dim.	Multimodal
Kalman/EKF	best (closed form)	no
Histogram/grid	worst (exponential)	yes
Particle filter	good (graceful)	yes

Histogram/grid: discretize the state space into cells, store P per cell. Prediction convolves the grid with the motion-model's uncertainty kernel; update multiplies each cell by the sensor likelihood, then renormalizes.

Easy to extract the mean or mode, and gives a well-understood approximation of the true posterior but the cell count grows exponentially with state dimension.

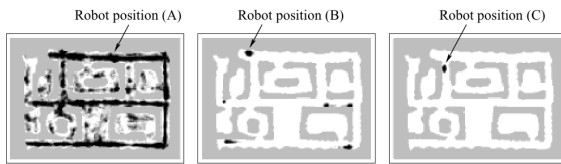


Figure 28: Global localization with a grid: the belief (dark = likely) starts smeared at (A), narrows at (B), and pins down a unique peak at (C) (cf. book Fig. 9.4).

Particle filter: represent the belief as a weighted set of N samples (x, ω) . Prediction samples each particle forward through the motion model; update reweights each particle by the sensor likelihood $P(y | x)$ (this follows from importance sampling: particles are drawn from the *predicted* belief rather than the true posterior, so each must be corrected by the target/proposal ratio, which reduces to exactly $P(y | x)$), then **resamples** — drawing N new particles with replacement, probability proportional to weight, a "survival of the fittest" step that concentrates particles where the posterior is largest.

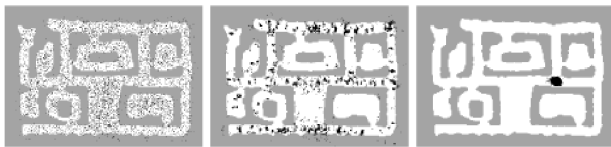


Figure 29: Global localization with a particle filter (10,000 samples): particles start spread uniformly over free space (left), thin out and cluster after fusing 10 sensor readings (middle), and collapse onto the true pose after 65 steps (right) — resampling is what drives this convergence (cf. book Fig. 9.7).

Systematic (low-variance) resampling: rather than drawing N independent samples from the weight distribution (**roulette-wheel** sampling, which has higher variance), lay the N weights end-to-end on $[0, 1)$, pick one random offset $r \in [0, 1/N)$, then take the particle under each of $r, r+1/N, r+2/N, \dots$ — a single sweep, $O(N)$ time, lower variance than N independent draws.

Monte Carlo Localization (MCL): the particle filter above, named for its most common use — robot localization. Per step: (1) sample each particle forward through the motion model given the executed control; (2) weight it by the sensor likelihood of the actual reading at that pose; (3) resample N particles proportional to weight. Exactly the recipe above, applied to localization specifically.

When to prefer each: Kalman/EKF when the belief stays comfortably unimodal and the state is high-dimensional — it is by far the cheapest. A histogram/grid when the state space is low-

dimensional (2–3D pose) and a full, high-accuracy posterior is needed, e.g. global localization from scratch. A particle filter when the belief may be multimodal *and* the space is too high-dimensional for a grid — the general-purpose default for real-world global localization, at the cost of being a Monte Carlo approximation (too few particles risks losing the true hypothesis entirely).

10.3 Mapping

Occupancy grid mapping: *given known robot poses*, estimate $P(\text{occupied})$ independently for every workspace cell via Bayesian updating from an inverse sensor model — essentially many small, independent Bayes-filter updates running in parallel, one per cell.

Mapping with known poses is comparatively easy. The hard problem is when the poses *aren't* known: without a map the robot cannot localize, and without localizing it cannot build a map. This chicken-and-egg problem is **SLAM**.

Bayesian SLAM estimates the joint posterior over the *entire* trajectory and the map at once, $P(x(1:k), m | u(0:k-1), y(1:k))$, via the same prediction/update recursion as §10.1. Two broad families tackle it:

- **EKF-SLAM** (§9.4): a sound, closed-form joint Gaussian over pose and landmarks, but essentially landmark-based and unimodal.
- **Global optimization / scan matching:** treat poses as random variables linked by constraints from overlapping measurements (a spring network to relax), or greedily align each new sensor scan to the map built so far (iterative closest point). This scales to dense, feature-rich maps without the $O(n_t^2)$ growth of EKF-SLAM, but batch optimization (e.g. EM) needs multiple passes over the whole data set, while pure greedy scan matching cannot revise a past pose once fixed — so accumulated error can leave large loops unclosed and the map globally inconsistent.

Prefer EKF-SLAM for a modest number of discrete, uniquely-identifiable landmarks where a compact Gaussian suffices. **Prefer particle-filter or optimization-based Bayesian SLAM** for large, dense, feature-rich maps where landmarks aren't naturally discrete and the trajectory belief may be multimodal — worth the extra computation for a representation that can close loops and stay globally consistent.